HW 4.3

AUTHOR
Anish Chitnis

PUBLISHED
April 15, 2026

1 Busiest Airports Analysis

This section looks at passenger traffic across six of the world's busiest airports from 2020 to 2025. The data was pulled from Wikipedia's List of Busiest Airports by Passenger Traffic page and cleaned up using R. The six airports we're looking at are Hartsfield-Jackson Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Dubai (DXB), Tokyo Haneda (HND), and Dallas/Fort Worth (DFW).

1.1 Passenger Traffic Table

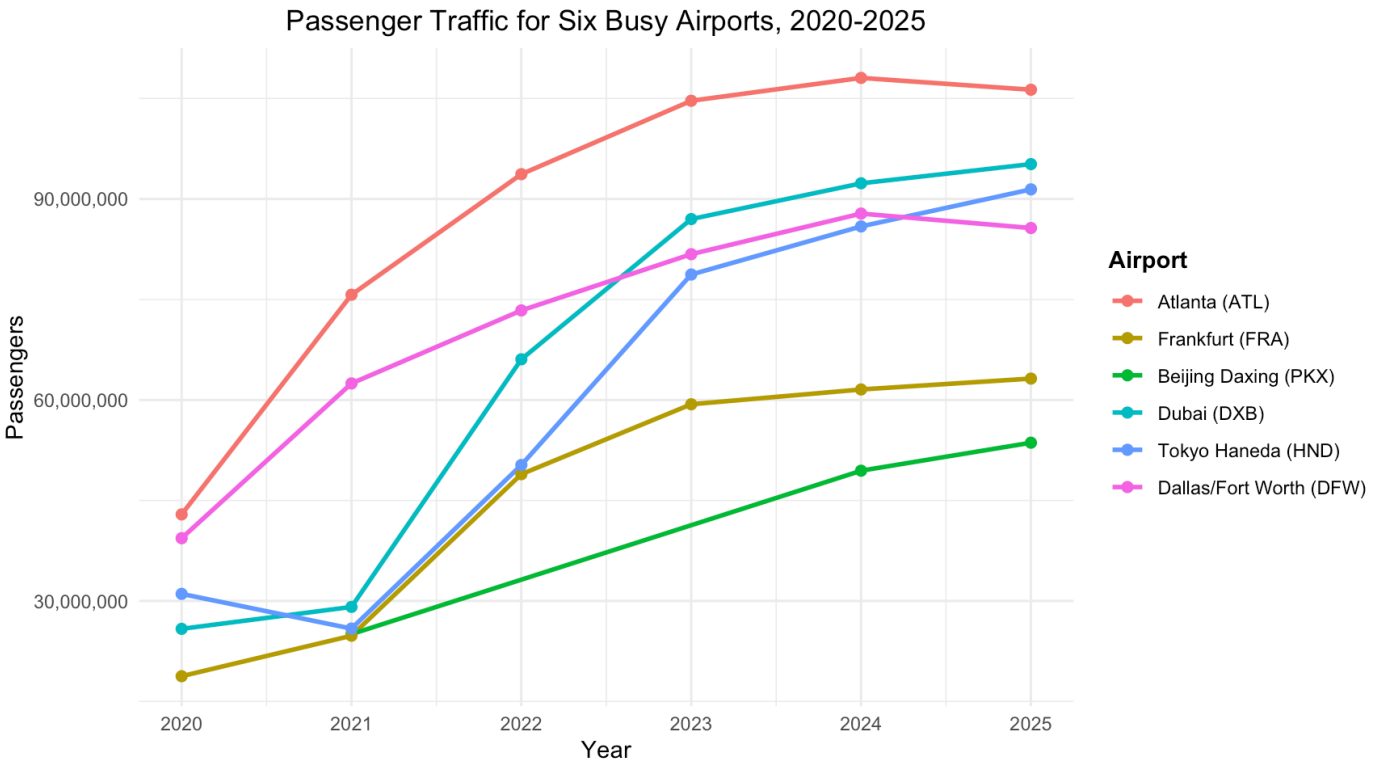
The table below shows the annual passenger counts for each airport. It's useful when you want to look up or compare exact numbers between specific airports and years.

Annual Passenger Traffic (in Millions) for Six Major Airports, 2020-2025

Airport	2020	2021	2022	2023	2024	2025
Beijing Daxing (PKX)	25.05	49.44	53.62	NA	NA	NA
Dallas/Fort Worth (DFW)	62.47	87.82	85.66	39.36	73.36	81.76
Dubai (DXB)	29.11	92.33	95.20	25.84	66.07	86.99
Frankfurt (FRA)	24.81	61.56	63.19	18.77	48.92	59.36
Atlanta (ATL)	75.70	108.07	106.30	42.92	93.70	104.65
Tokyo Haneda (HND)	25.88	85.90	91.43	31.06	50.29	78.72

Atlanta (ATL) sits at the top every single year, which makes sense given its role as a major connecting hub. Beijing Daxing (PKX) has missing values for 2020 through 2022, most likely because the airport was still ramping up operations and China had strict travel restrictions during the pandemic. The table is great for pinpointing exact numbers, but it's a bit hard to spot trends at a glance — that's where the plot comes in.

1.2 Passenger Traffic Plot



The line graph makes trends much easier to see. Every airport hit its lowest point around 2020-2021, which lines up with the COVID-19 pandemic grounding air travel globally. From 2022 onward, there's a sharp recovery across the board. Atlanta continues to lead the pack, while Dubai and Dallas both show strong growth heading into 2025. Frankfurt and Tokyo recover more gradually by comparison. Together, the table and plot give a complete picture — the plot tells the story over time, and the table lets you dig into specific numbers.

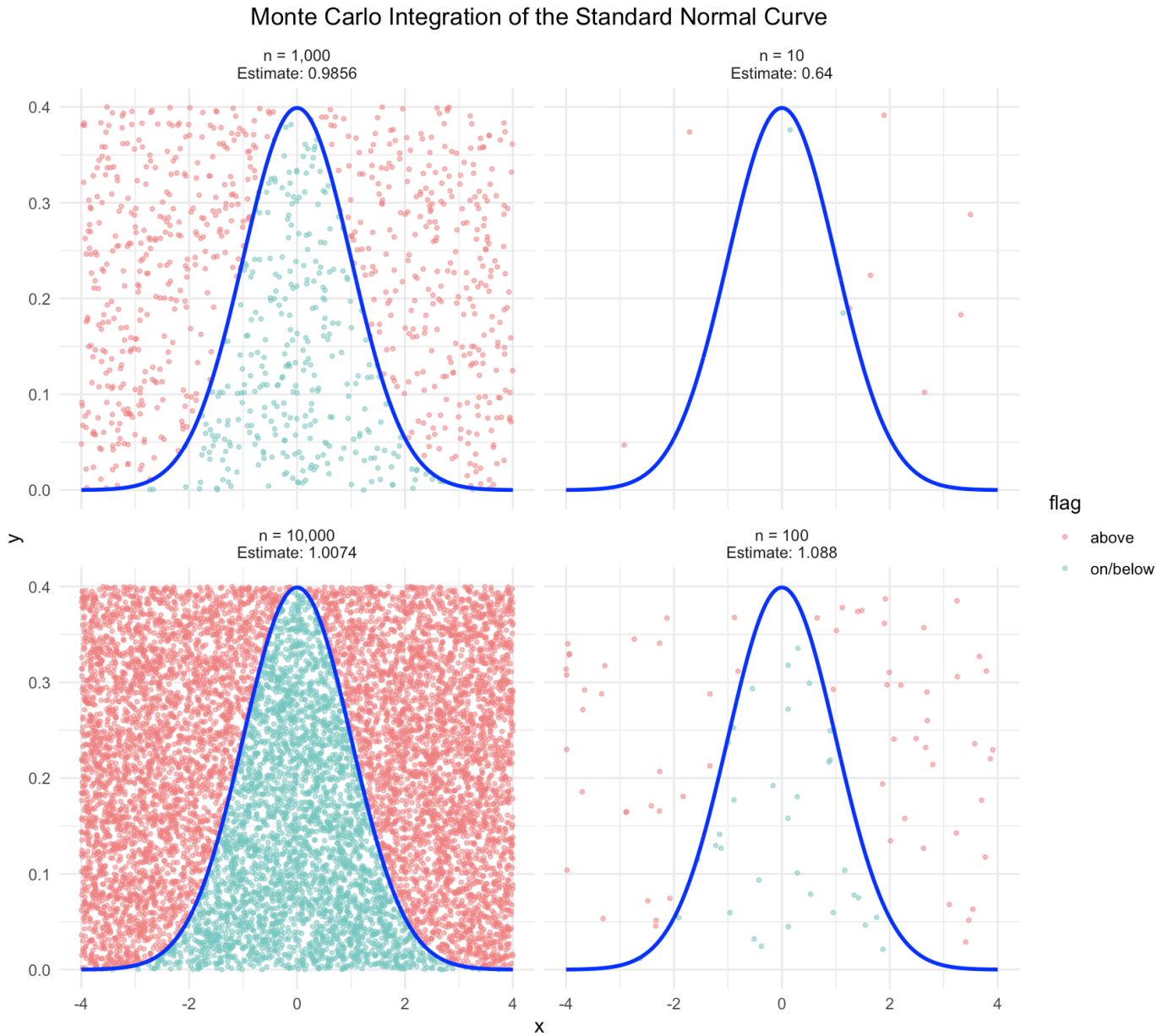
---

2 Monte Carlo Numerical Integration

Monte Carlo integration estimates the area under a curve by randomly throwing points at a bounding rectangle and seeing what fraction land underneath the curve. The more points you use, the closer the estimate gets to the true value.

Here I applied this method to estimate the integral of the standard normal distribution from -4 to 4. The true value of this integral is essentially 1, since the normal distribution integrates to 1 across its full range.

2.1 Small Multiples Visualization



The four panels make the pattern really clear. With only 10 points, the estimate is pretty far off — there just aren't enough darts to get a reliable read. At 100 it's better, but still a bit noisy. By 1,000 and especially 10,000 points, the estimate is very close to 1. This demonstrates one of the core ideas of Monte Carlo methods: more random samples means a better estimate. It scales well to problems that are otherwise very hard to solve analytically, and based on the simulation we can confidently say the integral of the standard normal curve from -4 to 4 is approximately 1.

---

3 Planning and Prompting GenAI Tools

3.1 The Data Context

The dataset for this section is `calcium.csv`, which comes from a longitudinal study that followed 31 women over four years. Researchers measured the calcium levels in the ulna of each woman's dominant arm using photon absorptiometry. Fifteen women served as a null treatment group and didn't receive any special intervention. The

other 16 women participated in a diet and exercise program for the duration of the study. Measurements were taken at the start and once per year for three additional years (Initial, Year 1, Year 2, Year 3). In the CSV, the null treatment group occupies the first four columns and the diet/exercise group occupies the last four.

3.2 My Plan

Before sending anything to a genAI tool, I looked at the actual data and thought through what needed to happen. Here's the plan i came up with:

Step 1: The file has 8 columns with duplicate names (y0, y1, y2, y3 repeated twice). That means the two groups are sitting side-by-side rather than stacked. I need to split the file into two halves: columns 1-4 for one group and columns 5-8 for the other.

Step 2: The data is in wide format, which doesn't work well for ggplot. I need to reshape it to long format so each row is one observation with columns for patient ID, group, time point, and bone density value.

Step 3: The groups have different numbers of patients (15 vs. 16), so there will be at least one NA row after combining. I need to drop those cleanly after pivoting.

Step 4: Once tidy, I want a line graph with time on the x-axis, bone density on the y-axis, individual patient trajectories as thin semi-transparent lines, and a thicker group mean line on top for each group. That way you can see both the individual variation and the overall trend at the same time.

3.3 The Prompt I Used

Here is the exact prompt I sent to Claude (claude.ai):

> "I have a CSV file called calcium.csv that has bone density data. The file has 8 columns — y0, y1, y2, y3, y0, y1, y2, y3 — where the first four columns are for a calcium treatment group and the last four are for a control group. The columns are repeated and there's no group label in the file itself.
>
> Here's what I want to do: (1) Read in the data and split it into the two groups, (2) reshape it from wide to long format so each row is one measurement with columns for patient ID, group, time point, and value, (3) handle any missing values since the groups might have different numbers of patients, and (4) make a ggplot line graph showing bone density over time for both groups, with individual patient lines and a thicker average line on top.
>
> Can you write the R code to do all of this using tidyverse?"

3.4 Plan-Based Prompt Response

Claude returned the following comlpete R script:

```r
library(tidyverse)

# 1. Read & split
raw <- read_csv("calcium.csv", show_col_types = FALSE)

# First four columns = calcium group, last four = control group
calcium_wide <- raw |> select(1:4) |> rename(y0 = 1, y1 = 2, y2 = 3, y3 = 4)
control_wide <- raw |> select(5:8) |> rename(y0 = 1, y1 = 2, y2 = 3, y3 = 4)

# 2. Reshape to long format
to_long <- function(df, group_label) {
  df |>
    mutate(patient_id = row_number()) |>
    pivot_longer(
      cols      = starts_with("y"),
      names_to  = "time_point",
      values_to = "bone_density"
    ) |>
    mutate(
      group      = group_label,
      time_point = as.integer(str_remove(time_point, "y"))
    )
}

# 3. Handle missing values
long_data <- bind_rows(
  to_long(calcium_wide, "Calcium"),
  to_long(control_wide, "Control")
) |>
  drop_na(bone_density) |>
  mutate(group = factor(group, levels = c("Calcium", "Control")))

# 4. Compute group-level averages
avg_data <- long_data |>
  summarise(
    mean_density = mean(bone_density),
    .by = c(group, time_point)
  )

# 5. Plot
group_colors <- c("Calcium" = "#2196F3", "Control" = "#F44336")

ggplot() +
  geom_line(
    data    = long_data,
    mapping = aes(
      x     = time_point,
      y     = bone_density,
      group = interaction(patient_id, group),
      color = group
```

```
    ),
    alpha = 0.35, linewidth = 0.4
  ) +
  geom_line(
    data    = avg_data,
    mapping = aes(x = time_point, y = mean_density, color = group, group = group),
    linewidth = 1.8
  ) +
  geom_point(
    data    = avg_data,
    mapping = aes(x = time_point, y = mean_density, color = group),
    size = 3, shape = 21, fill = "white", stroke = 1.5
  ) +
  scale_color_manual(values = group_colors, name = "Group") +
  scale_x_continuous(breaks = 0:3, labels = paste0("Year ", 0:3)) +
  labs(
    title    = "Bone Density Over Time by Treatment Group",
    subtitle = "Thin lines = individual patients · Thick lines = group mean",
    x        = "Time Point",
    y        = "Bone Density",
    caption  = "Calcium group: n = 15 · Control group: n = 16"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title         = element_text(face = "bold", size = 15),
    plot.subtitle      = element_text(color = "grey45", size = 10),
    legend.position    = "top",
    panel.grid.minor   = element_blank(),
    panel.grid.major.x = element_blank()
  )
```

3.5 Generic Prompt Response

The following is the response generated when Claude was given only the generic prompt:

> **Data Structure:** The CSV has 8 columns — the first four belong to the null treatment group (n=15) and the last four to the diet/exercise group (n=16). Each set of four columns represents ulnar calcium measurements at Initial, Year 1, Year 2, and Year 3.
>
> **Group Means Over Time:**

Timepoint	Null Group	Diet/Exercise Group
Initial	72.38	69.29
Year 1	73.29	70.66
Year 2	72.47	71.18

Timepoint	Null Group	Diet/Exercise Group
Year 3	64.79	64.53

Mean Change from Initial:

Timepoint	Null Group	Diet/Exercise Group
Year 1	+0.91	+1.37
Year 2	+0.09	+1.89
Year 3	-7.59	-4.76

Both groups decline sharply by Year 3, consistent with natural bone loss over time. The diet/exercise group loses less calcium by Year 3 (-4.76 vs. -7.59), suggesting the intervention may have a modest protective effect. Years 1 and 2 show slight gains or stability before the Year 3 drop.

3.6 Comparing the Two Approaches

The difference between the two responses is pretty striking. The plan prompt came back with a complete, working R script, a full ggplot visualization, and specific notes on how the data quirks were handled. The generic prompt came back with a summary table and some observations.

That gap makes sense when you think about it. Because my prompt described exactly what the data looked like and what I wanted to produce, Claude could go straight to building it. The generic prompt gave Claude a description but no direction, so it defaulted to a high-level summary.

The other clear difference is how each handled the data issues. Because I mentioned the duplicate column names and unequal group sizes upfront, Claude addressed both correctly without any follow-up. The generic response didn't attempt to wrangle or visualize the data at all. That's probably the strongest argument for planning before prompting.

---

4 Reflection

Looking back at what we've covered so far, the biggest shift for me has been in how I think about data before I ever write a line of code. Early on I'd just load something in and figure it out as I went. Now I actually look at the structure first, think about what format I need, and make a plan. Xeactly like I did with the calcium dataset above.

A few things that really clicked for me:

The tidyverse philosophy of tidy data genuinely changed how I approach problems. Once data is in long format with one observation per row, everything else flows naturally. The airports assignment was a good example. The Wikipedia data needed quite a bit of reshaping before it was usable, and having `pivot_longer`, `pivot_wider`, and `left_join` in my toolkit made that feel manageable instead of overwhelming.

Quarto has been a big deal for reproducibility. Before this course my analysis would live in scattered R scripts with notes in a separate document. Quarto puts everything in one place and the ability to hide code from the main document while keeping it in an appendix is exactly how I'd want to share work with someone who doesn't care about implementation details.

Monte Carlo was probably the most conceptually interesting thing we've done. It's such a simple idea but it actually works, and watching the estimate converge as n grows was genuinely satisfying. It also drove home why reproducibility matters.

Overall, I feel like I'm starting to think like a data scientist rather than just someone who knows some R syntax, and that's a meaningful difference.

---

5 Appendices

5.1 Appendix A: GenAI Usage

Tool: Claude (Anthropic), claude.ai — Claude Sonnet 4
Date of Use: April 15, 2026

Exact Prompt Used:

> "I have a CSV file called calcium.csv that has bone density data. The file has 8 columns — y0, y1, y2, y3, y0, y1, y2, y3 — where the first four columns are for a calcium treatment group and the last four are for a control group. The columns are repeated and there's no group label in the file itself.
>
> Here's what I want to do: (1) Read in the data and split it into the two groups, (2) reshape it from wide to long format so each row is one measurement with columns for patient ID, group, time point, and value, (3) handle any missing values since the groups might have different numbers of patients, and (4) make a ggplot line graph showing bone density over time for both groups, with individual patient lines and a thicker average line on top.
>
> Can you write the R code to do all of this using tidyverse?"

Exact Response:

Claude confirmed the data structure (16 rows, 8 columns; calcium group n=15 after dropping one fully-NA row, control group n=16) and returned the complete R script in Section 3.4 above.

Generic Prompt (provided by instructor):

These data come from a longetudinal study that followed 31 women over the course of four years. At the start of the study, researchers measured the amount of calcium in the ulna from each woman's dominant arm through photon absorptiometry. Throughout the next four years, 15 of the women went about their lives as they typically

did; they make up the null treatment group haivng not received any special treatment. The other 16 women spent the next four years receiving special help with their diet and completing a regular exercise program; they make up the diet/exercise treatment group. Once a year each of thee 31 women had their ulnar calcium levels measured. In the calcium.csv file, you'll find the measurements (Initial, Year 1, Year 2, Year 3) for each woman; the null treatment group take up the first four columns and the diet/exercise group make up the second four columns.

Exact Response to Generic Prompt:

Claude returned a summary table of group means at each time point and mean changes from baseline, along with three observations: both groups decline sharply by Year 3; the diet/exercise group loses less calcium by Year 3 (-4.76 vs. -7.59); and Years 1 and 2 show slight stability before the Year 3 drop. No code or visualization was produced. The full response is is Section 3.5 above.

5.2 Appendix B: Code Appendix

All code chunks from this document are displayed below.

```
# Load all necessary packages
library(tidyverse)
library(rvest)
library(knitr)
library(kableExtra)
# Reference table mapping IATA codes to full airport names
airport_ref <- tibble(
  code_tag = c("ATL", "FRA", "PKX", "DXB", "HND", "DFW"),
  airport_name = c(
    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "Dubai International Airport",
    "Tokyo Haneda Airport",
    "Dallas Fort Worth International Airport"
  )
)

# Helper to standardize column names to snake_case
clean_col_names <- function(col_names) {
  col_names %>%
    str_to_lower() %>%
    str_replace_all("[^a-z0-9]+", "_") %>%
    str_replace_all("^_|_$", "")
}

# Extract passenger count and airport code for a given year's Wikipedia table
pull_year_data <- function(one_table, yr) {
  names(one_table) <- clean_col_names(names(one_table))
  code_field <- names(one_table)[str_detect(names(one_table), "^code")][1]
```

```
  pax_field  <- names(one_table)[str_detect(names(one_table), "total.*passengers|passenge
  one_table %>%
    transmute(
      year            = as.integer(yr),
      code_tag        = str_extract(.data[[code_field]], "^[A-Z]{3}"),
      passenger_count = parse_number(.data[[pax_field]])
    )
}

# Scrape Wikipedia tables for years 2020-2025
wiki_link  <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffi
all_tables <- read_html(wiki_link) %>%
  html_elements("table.wikitable") %>%
  html_table()

# Combine all years, filter to our 6 airports, join full names
tidy_airports <- map2(
  all_tables[1:6],
  2025:2020,
  pull_year_data
) %>%
  list_rbind() %>%
  filter(code_tag %in% airport_ref$code_tag) %>%
  left_join(airport_ref, by = "code_tag") %>%
  select(year, code_tag, airport_name, passenger_count) %>%
  arrange(airport_name, year)
# Reshape to wide format with short labels; convert counts to millions for readability
table_wide <- tidy_airports %>%
  mutate(
    label = case_when(
      code_tag == "ATL" ~ "Atlanta (ATL)",
      code_tag == "FRA" ~ "Frankfurt (FRA)",
      code_tag == "PKX" ~ "Beijing Daxing (PKX)",
      code_tag == "DXB" ~ "Dubai (DXB)",
      code_tag == "HND" ~ "Tokyo Haneda (HND)",
      code_tag == "DFW" ~ "Dallas/Fort Worth (DFW)"
    ),
    passenger_millions = round(passenger_count / 1e6, 2)
  ) %>%
  select(year, label, passenger_millions) %>%
  pivot_wider(names_from = year, values_from = passenger_millions)

# Render styled table
table_wide %>%
  kable(
    col.names = c("Airport", as.character(2020:2025)),
    caption   = "Annual Passenger Traffic (in Millions) for Six Major Airports, 2020-2025
    align     = c("l", rep("r", 6))
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
```

```
    full_width        = FALSE
  )
# Set factor levels and labels for consistent legend ordering
plot_ready_airports <- tidy_airports %>%
  mutate(
    airport_name = factor(
      airport_name,
      levels = c(
        "Hartsfield-Jackson Atlanta International Airport",
        "Frankfurt Airport",
        "Beijing Daxing International Airport",
        "Dubai International Airport",
        "Tokyo Haneda Airport",
        "Dallas Fort Worth International Airport"
      ),
      labels = c(
        "Atlanta (ATL)",
        "Frankfurt (FRA)",
        "Beijing Daxing (PKX)",
        "Dubai (DXB)",
        "Tokyo Haneda (HND)",
        "Dallas/Fort Worth (DFW)"
      )
    )
  )

# Line plot with points per airport per year
ggplot(plot_ready_airports, aes(x = year, y = passenger_count, color = airport_name)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(
    title = "Passenger Traffic for Six Busy Airports, 2020-2025",
    x     = "Year",
    y     = "Passengers",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    plot.title   = element_text(hjust = 0.5),
    legend.title = element_text(face = "bold")
  )
# Core Monte Carlo function: throws random darts at a bounding rectangle
# and flags each as on/below or above the curve
mc_simulation <- function(curve_fn, sample_count, x_lower, x_upper, y_lower, y_upper) {
  dart_x       <- runif(sample_count, min = x_lower, max = x_upper)
  dart_y       <- runif(sample_count, min = y_lower, max = y_upper)
  curve_height <- curve_fn(dart_x)
  hit_status   <- ifelse(dart_y <= curve_height, "on/below", "above")
```

```r
  data.frame(x = dart_x, y = dart_y, f_x = curve_height, flag = hit_status)
}

# Standard normal density as the target curve
normal_curve <- function(x) dnorm(x, mean = 0, sd = 1)

# Area of bounding rectangle: x in [-4,4], y in [0, 0.4]
rect_area <- (4 - (-4)) * (0.4 - 0)

# Run simulation for a given n; returns data and area estimate
run_mc <- function(n) {
  sim      <- mc_simulation(normal_curve, n, -4, 4, 0, 0.4)
  estimate <- rect_area * sum(sim$flag == "on/below") / nrow(sim)
  list(data = sim, estimate = estimate, n = n)
}

# Set seed for reproducibility and run four sample sizes
set.seed(42)
results <- map(c(10, 100, 1000, 10000), run_mc)
# Combine all simulation runs into one data frame with a facet label per n
plot_data <- map_dfr(results, function(r) {
  r$data %>%
    mutate(
      label = paste0("n = ", scales::comma(r$n), "\nEstimate: ", round(r$estimate, 4))
    )
})

# Smooth curve overlay data replicated across all four facets
curve_data <- tibble(x = seq(-4, 4, length.out = 400), y = normal_curve(x))

# Small multiples plot: one panel per sample size
ggplot(plot_data, aes(x = x, y = y, color = flag)) +
  geom_point(alpha = 0.5, size = 0.8) +
  geom_line(
    data        = curve_data %>% crossing(label = unique(plot_data$label)),
    mapping     = aes(x = x, y = y),
    color       = "blue",
    linewidth   = 1,
    inherit.aes = FALSE
  ) +
  scale_color_manual(values = c("above" = "#F08080", "on/below" = "#76C7C0")) +
  facet_wrap(~label, ncol = 2) +
  labs(
    title = "Monte Carlo Integration of the Standard Normal Curve",
    x     = "x",
    y     = "y",
    color = "flag"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```